

Study_Case1



4143108 - Software Architecture and Design

Nama: Yos Ivan Sirait

NIM: 11423025

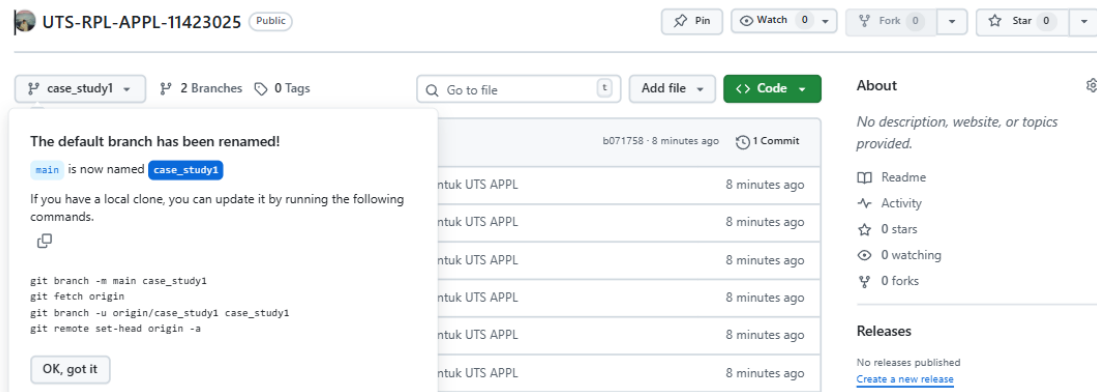
Sarjana Terapan Teknologi Rekayasa

Perangkat Lunak

FAKULTAS VOKASI

INSTITUT TEKNOLOGI DEL

Case Study 1 : "MediTrack - A Digital Healthcare Platform"



1. Arsitektur yang Dipilih

Berdasarkan kebutuhan platform dan ekspektasi pertumbuhan, saya memilih arsitektur Monolitik Modular menggunakan Next.js (App Router) sebagai framework full-stack dan Supabase sebagai Backend-as-a-Service (PostgreSQL + Auth + Storage). Alasan utama: kesederhanaan, kecepatan pengembangan, dan integrasi yang kuat dengan Supabase.

a. Justifikasi dengan Prinsip Arsitektur

1. Kesederhanaan (Simplicity)

- Satu codebase memudahkan pengembangan, debugging, dan deployment.
- Next.js menyediakan API routes dan serverless functions yang terintegrasi, sehingga tim dapat fokus pada fitur tanpa kompleksitas jaringan.

2. Kecepatan Pengembangan (Development Velocity)

- Perubahan lintas modul dapat dilakukan dengan cepat karena semua kode berada dalam satu proyek.
- Supabase mempercepat development dengan menyediakan autentikasi, realtime, dan storage siap pakai.

3. Konsistensi dan Integritas Data (Consistency)

- Menggunakan satu database (Supabase) memastikan transaksi ACID untuk operasi yang melibatkan banyak modul (misal booking appointment yang memerlukan update EHR dan pembayaran).
- Penting untuk data medis yang membutuhkan integritas tinggi.

b. Trade-offs dan Keterbatasan

- Skalabilitas horizontal terbatas – karena satu aplikasi, scaling harus dilakukan pada level instance (load balancing) bukan per modul
- Keterikatan modul – perubahan pada satu modul berpotensi mempengaruhi modul lain. Untuk mengurangi risiko, diterapkan modularisasi dalam struktur folder dan pemisahan tanggung jawab.
- Teknologi terikat – semua harus menggunakan stack yang sama (Next.js, Supabase), tidak bisa memilih teknologi berbeda per modul.

2. System Decomposition & Modeling

A. Tabel berikut merangkum modul utama beserta tanggung jawabnya:

Modul	Tanggung Jawab	Entity Utama
IAM (Identity)	Registrasi, Login, dan RBAC (Role-Based Access Control)	User, Role
Appointment	Janji Temu	Appointment, doctor schedule
EHR	Menyimpan Riwayat medis	MedicalRecord
Pharmacy	Pengelolaan resep digital dan stok obat	Medication Stock
Payment	Payment Gateway	Payment
Analytics	AnalyticsPelaporan & Business Intelligence	AnalyticsPelaporan & Business IntelligenceVisualisasi KPI rumah sakit, performa

		dokter, dan tren kesehatan pasien.
--	--	------------------------------------

B. Struktur Modular

Break down the MediTrack system into core modules or services based on your chosen architecture.

```

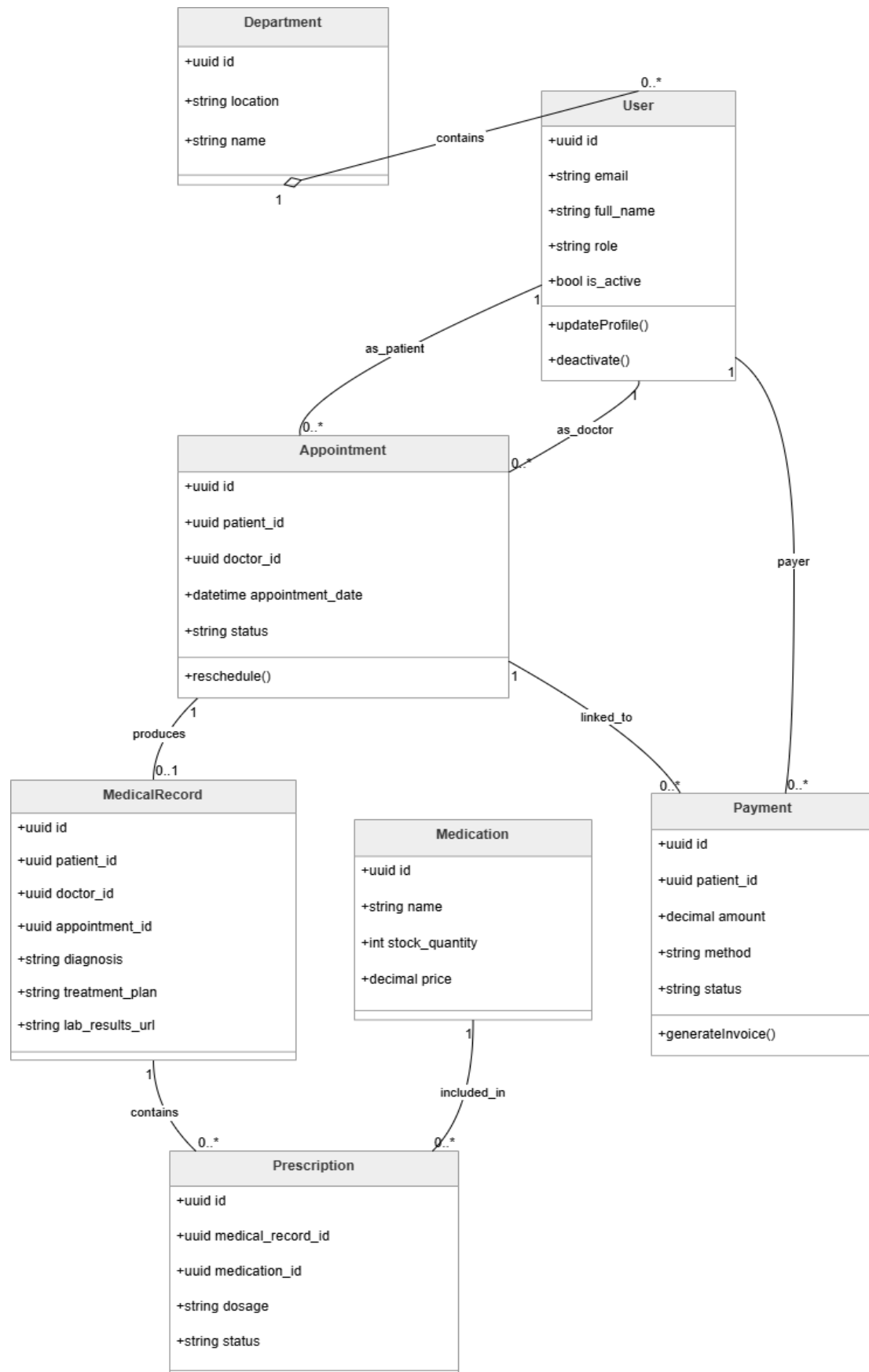
app/
  api/
    auth/
    users/
    appointments/
    medical-records/
    pharmacy/
      medications/
      prescriptions/
    payments/
    analytics/
  lib/
    auth.ts
    db.ts
    validators.ts
  types/
    index.ts
  scripts/
    01-init-schema.sql

```

c. Relationships Diagram



d. Use UML class diagrams or component diagrams to illustrate the design



3. Archetecture Visualization

a.



Batasan Arsitektur dan Interaksi Sistem

1. **Batasan Modular (Modular Boundaries)** Dalam aplikasi Next.js ini, setiap modul (IAM, Appointment, EHR, dll.) dipisahkan secara logis ke dalam direktorinya masing-masing di bawah `lib/services/` atau `app/api/`. Hal ini memastikan bahwa logika bisnis terenkapsulasi di dalam modulnya sendiri. Komunikasi antar-modul ditangani melalui panggilan layanan internal atau antarmuka bersama (*shared interfaces*) yang terdefinisi dengan baik. Pendekatan ini mencegah akses database lintas-modul secara langsung sedapat mungkin untuk menjaga integritas modular.

2. **Batasan Lapisan Layanan (Service Layer Boundaries)** Lapisan layanan (*Service Layer*) bertindak sebagai pusat logika bisnis inti. Lapisan ini mengabstraksi akses data yang mendasari (Supabase/PostgreSQL) dan integrasi eksternal, sehingga menyediakan API yang bersih untuk dikonsumsi oleh frontend dan rute API (*API routes*). Pemisahan ini memastikan

bahwa perubahan pada skema database atau penyedia layanan eksternal tidak berdampak langsung pada antarmuka pengguna (User Interface).

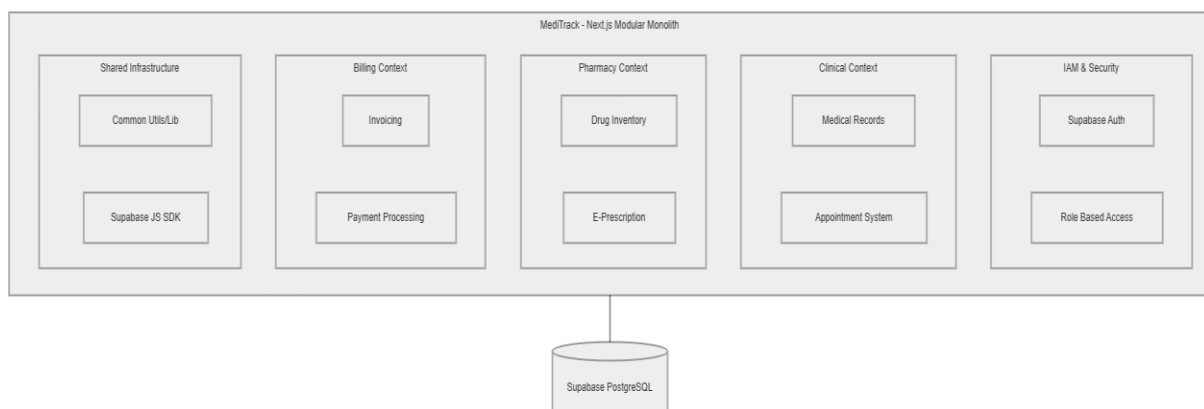
3. Aliran Data (Data Flow)

Berikut adalah urutan aliran data dalam sistem MediTrack:

1. Aksi Pengguna: Pengguna berinteraksi dengan Web Client (misalnya, melakukan pemesanan janji temu).
2. Permintaan (Request): Frontend mengirimkan permintaan ke rute API (API Route) yang sesuai.
3. Logika Bisnis: Rute API memanggil Layanan (Service) terkait (misalnya, Appointment Service).
4. Penyimpanan/Pengambilan Data: Layanan berinteraksi dengan Supabase Auth untuk identitas atau Database PostgreSQL untuk operasi data.
5. Integrasi Eksternal: Jika diperlukan, Layanan berkomunikasi dengan Sistem Eksternal (misalnya, memproses pembayaran melalui gateway).
6. Respons: Hasil dikirimkan kembali melalui Rute API ke Web Client, yang kemudian memperbarui tampilan antarmuka (UI).

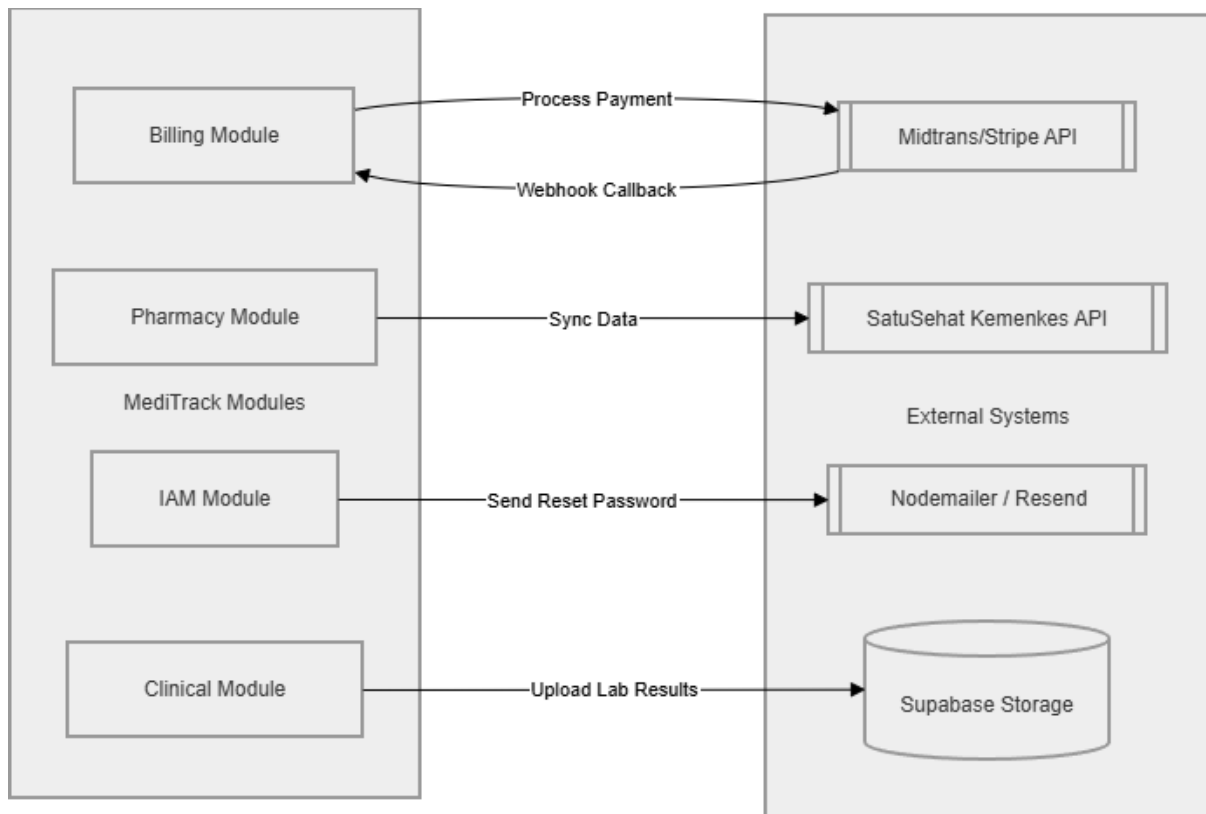
b. Domain Boundaries (Bounded Contexts)

Bagian ini menjelaskan bagaimana fitur-fitur rumah sakit yang dipisahkan secara logika meskipun berada dalam satu aplikasi.



c. External Systems Integration

Bagian ini menjelaskan bagaimana modul di dalam aplikasi kamu berinteraksi dengan layanan luar (seperti Payment Gateway atau API lainnya).



d. Performance Bottlenecks & Mitigation Strategies

Sebagai sistem yang dipersiapkan untuk pertumbuhan cepat, beberapa potensi hambatan kinerja telah diidentifikasi beserta strategi mitigasinya:

- Database Connection Overload: Karena menggunakan arsitektur monolitik dengan banyak modul, beban pada satu database PostgreSQL bisa meningkat drastis.
 - ⇒ Mitigasi: Mengaktifkan Connection Pooling (menggunakan fitur Supabase/Prisma Accelerate) untuk x mengelola koneksi database secara efisien.

e. Implementasi

1. Dashboard Admin



admin
Admin [Logout](#)

Administration Dashboard

Manage system users, payments, and overall operations

Total Users
234

Total Revenue
\$12,450

Pending Payments
8

Users Management
Manage all system users
[Manage Users](#)

Payments
View and manage all payments
[View Payments](#)

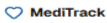
Medications
Manage medication inventory
[Manage Medications](#)

Departments
Manage hospital departments
[Manage Departments](#)

Analytics
View system analytics and reports
[View Analytics](#)

System Settings
Configure system settings
[Settings](#)

2. Dashoboard Doctor



Mikael Sitorus
Doctor [Logout](#)

Doctor Dashboard

Manage appointments and patient care

Today Appointments
5

Total Patients
42

Medical Records
150

My Appointments
View and manage your appointments
[Manage Appointments](#)

My Patients
View your patient list
[View Patients](#)

Prescriptions
Create and manage prescriptions
[Manage Prescriptions](#)

Medical Records
Create and view medical records
[Manage Records](#)

c. Admin Create User

← Back to Dashboard

MediTrack Admin

admin
Admin

Users Management

Manage all system users and their roles

+ Add User

All Users

Total users: 0

Name	Email
------	-------

Add New User

Create a new user account

Full Name

YOS IVAN SIRAIT

Email

ivansrt887@gmail.com

Password

Role

Doctor

Confirm Password

Confirm password

Phone

082162119108

Date of Birth

04 / 30 / 2002

Gender

Female

Cancel

Create User

Actions

Users Management

Manage all system users and their roles

+ Add User

All Users

Total users: 8

Name	Email	Role	Phone	Status	Actions
YOS IVAN SIRAIT	ivansrt883@gmail.com	patient	+6282162119108	Active	 
YOS IVAN Manurung	ivansrt884@gmail.com	doctor	+6282162119108	Active	 
YOS IVAN HUTAPEA	ivansrt885@gmail.com	patient	+6282162119108	Active	 
YOS IVAN HUTAPEA	ivansrt887@gmail.com	patient	+6282162119108	Active	 
admin	admin@gmail.com	admin	+6282162119199	Active	 
Apoteker	Apoteker@gmail.com	pharmacist	+6282162119100	Active	 
YOS IVAN SIRAIT	ivansrt8899@gmail.com	patient	+6282162119108	Active	 
MIKAEL SILALAHI	mikael@gmail.com	patient	+6282162119108	Active	 

e. Tambah Medication

The screenshot displays the 'Medications Management' interface. At the top, there's a header 'Medications Management' with a subtitle 'Manage medication inventory and stock levels' and a '+ Add Medication' button. Below the header, there's a table titled 'All Medications' with columns 'Name', 'Strength', and 'Actions'. A modal form titled 'Add New Medication' is open, allowing users to add a new medication to the inventory. The form includes fields for 'Name *', 'Generic Name', 'Strength', 'Unit', 'Form', 'Manufacturer', 'Price (\$)', 'Stock Quantity *', 'Reorder Level *', and a 'Description' text area. At the bottom of the modal, there are 'Cancel' and 'Create Medication' buttons.

Medications Management
Manage medication inventory and stock levels

Add New Medication
Add a new medication to inventory

Name *
Generic Name

Strength Unit Form

Manufacturer Price (\$)

Stock Quantity * Reorder Level *

Description

Cancel Create Medication

f. Appointments oleh User

The screenshot shows the 'My Appointments' interface. At the top, there's a header 'My Appointments' with a subtitle 'View and manage your scheduled appointments' and a '+ Book New Appointment' button. Below the header, there's a list of appointments. One appointment is shown for 'Senin, 20 April 2026' with a status of 'scheduled'. The appointment details include 'Duration: 30 minutes', 'Reason: sasasasa', and 'Notes: sasasa'. A red trash icon is visible next to the appointment.

My Appointments
View and manage your scheduled appointments

Senin, 20 April 2026 scheduled

Duration: 30 minutes

Reason: sasasasa

Notes: sasasa

g. Appointments kepada Doctor

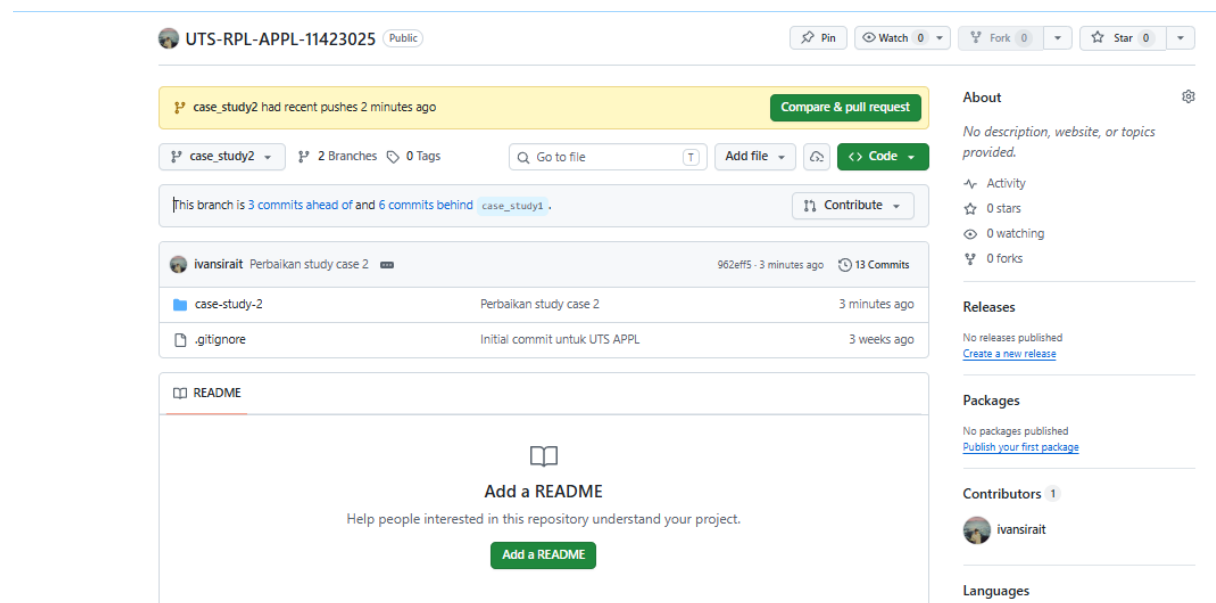
The screenshot shows the 'My Appointments' section of the MediTrack application. At the top, there is a 'Back to Dashboard' button. Below the title, a subtitle reads 'View and manage your patient appointments'. There are three summary cards: 'Scheduled' with a count of 1, 'Completed' with a count of 0, and 'Cancelled' with a count of 0. Below these cards, a detailed appointment entry is shown for 'Senin, 20 April 2026' with a status of 'Scheduled'. The patient is 'MIKAEL SILALAH', the duration is '30 minutes', the reason is 'sasasasa', and the notes are 'sasasa'. There are 'Mark Completed' and 'Cancel' buttons for this appointment.

h. Madications Management

The screenshot shows the 'Medications Management' section of the MediTrack Admin application. At the top, there is a 'Back to Dashboard' button and the user is logged in as 'admin Admin'. A subtitle reads 'Manage medication inventory and stock levels'. There is an 'Add Medication' button. Below this, there is a table for 'All Medications' with columns for 'Name', 'Strength', and 'Actions'. A modal window titled 'Add New Medication' is open, allowing the user to add a new medication to the inventory. The modal contains the following fields: 'Name *', 'Generic Name', 'Strength', 'Unit', 'Form', 'Manufacturer', 'Price (\$)', 'Stock Quantity *', 'Reorder Level *', and 'Description'. There are 'Cancel' and 'Create Medication' buttons at the bottom of the modal.

Study_Case2

1. Soal Ini dikerjakan di Brach 2

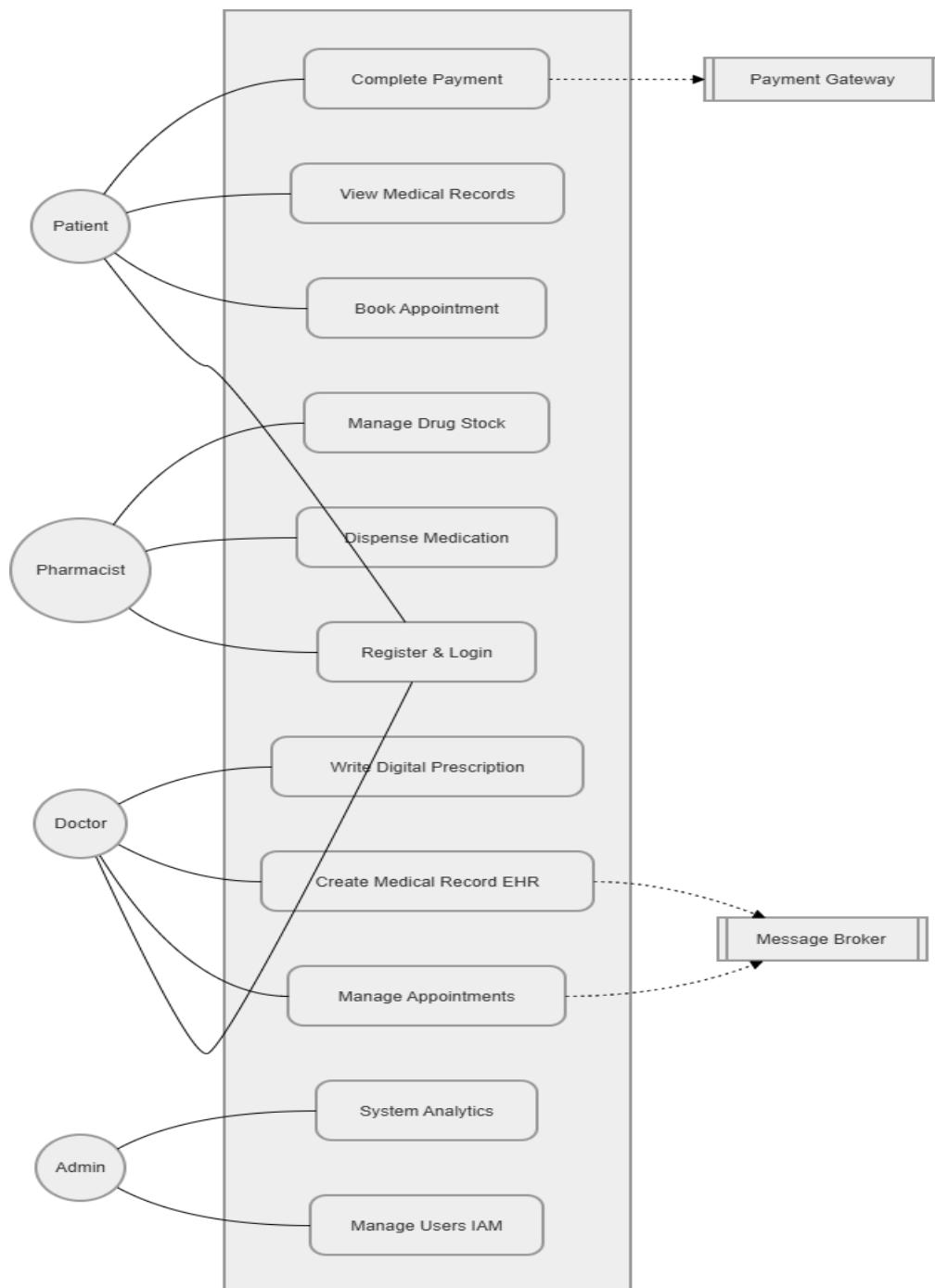


2. Use Case Analysis

Use Case Analysis MediTrack adalah platform layanan kesehatan yang mengelola alur kerja klinis. Berikut adalah ringkasan Use Case berdasarkan aktor utama:

- Patient (Pasien):** Melakukan registrasi/login, memesan janji temu (appointment), melihat rekam medis, dan melakukan pembayaran.
- Doctor (Dokter):** Mengelola jadwal janji temu (menerima/menolak), membuat rekam medis (EHR), dan menulis resep digital.
- Pharmacist (Apoteker):** Mengelola stok obat dan memproses resep obat untuk pasien.
- Admin:** Mengelola data pengguna (IAM) dan memantau operasional sistem.

- e. External Systems: Payment Gateway (pembayaran), Notification Service (WA/Email), dan Message Broker (sinkronisasi data).

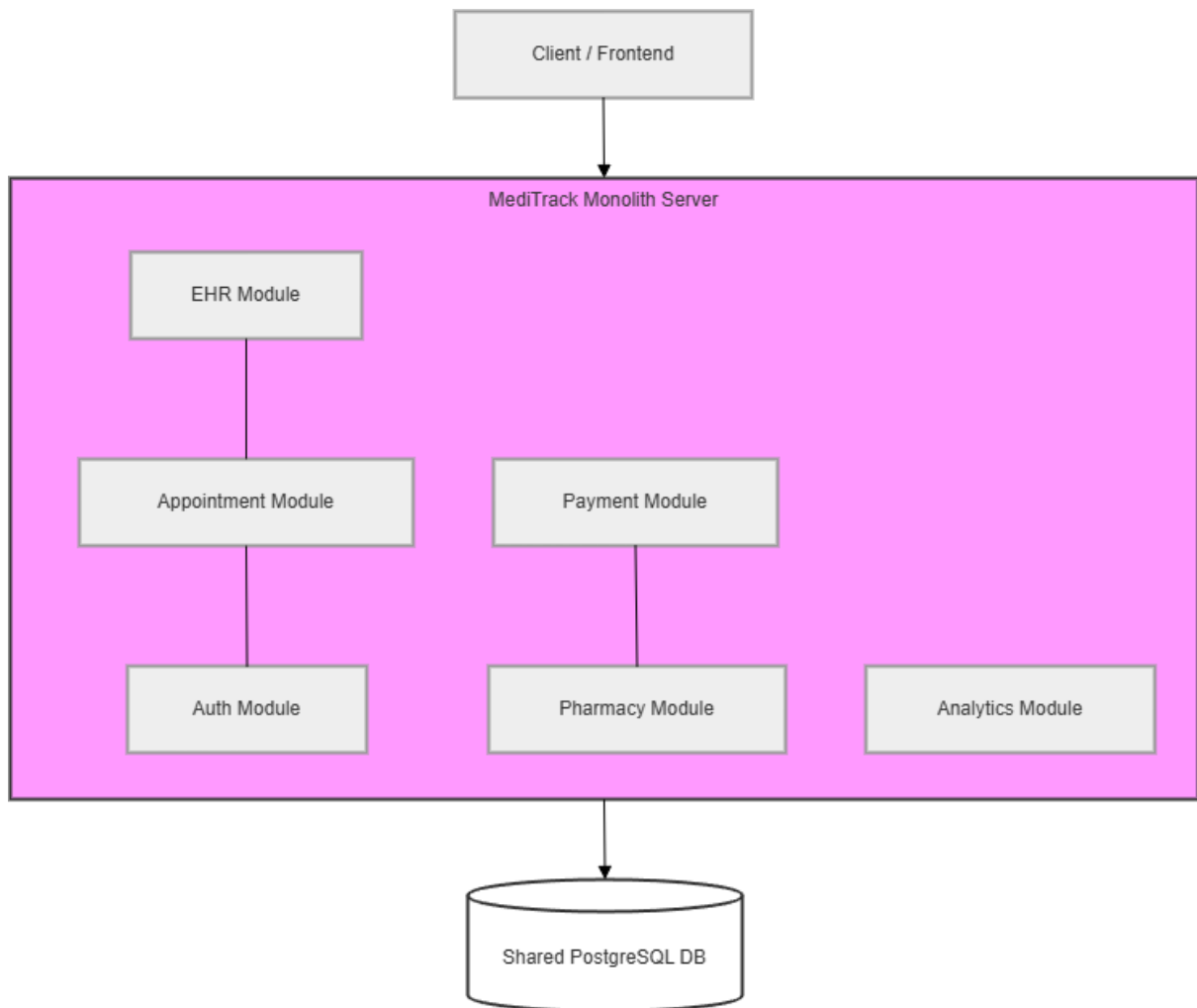


Monolith vs Microservices Comparison

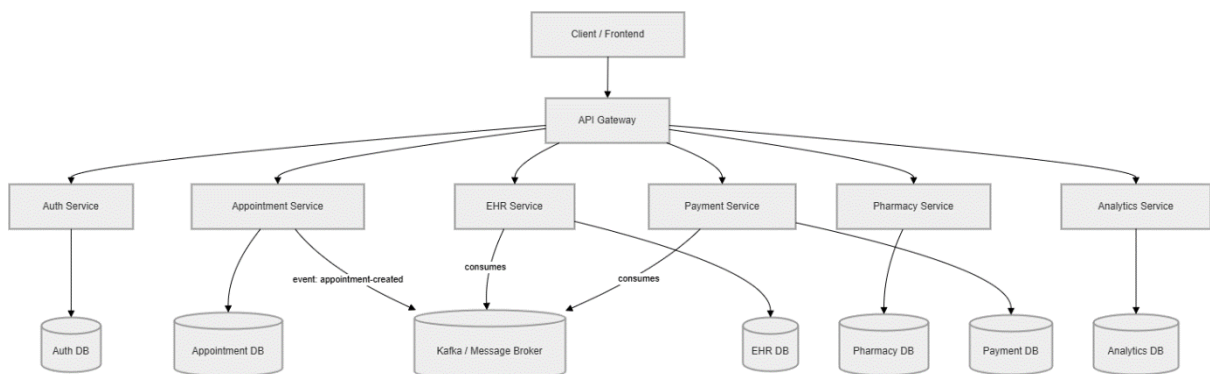
Dalam transformasi ini, fungsionalitas lama tidak dihapus, melainkan didistribusikan ke dalam layanan-layanan yang berdiri sendiri (standalone).

A. Monolith Architecture (Kondisi Awal):

- **Boundary:** Semua modul (Auth, EHR, Payment) berada dalam satu codebase dan satu runtime.
- **Data Flow:** Berbagi satu database yang sama (Shared Database). Interaksi antar modul menggunakan pemanggilan fungsi langsung (Direct Method Calls).
- **Integration:** Sulit diintegrasikan secara parsial karena ketergantungan yang tinggi.



B. Microservices Architecture (Hasil Transformasi):



3. Reasoning & Trade-offs

- a. Reasoning (Alasan Transformasi):
 - Scalability: Memungkinkan untuk melakukan scale-up hanya pada service yang sibuk (misal: Appointment Service saat jam ramai).
 - Fault Tolerance: Jika Pharmacy Service mati, layanan lain seperti Auth atau Appointment tetap bisa berjalan.
 - Independent Deployment: Tim pengembang dapat merilis fitur di EHR Service tanpa harus menunggu modul Payment selesai.
- b. Trade-offs (Konsekuensi):
 - Complexity vs Scalability: Sistem lebih skalabel, namun konfigurasi infrastruktur (deployment) menjadi jauh lebih rumit.
 - Consistency: Data tidak lagi langsung konsisten (Eventual Consistency) karena database terpisah, sehingga perlu mekanisme Message Broker

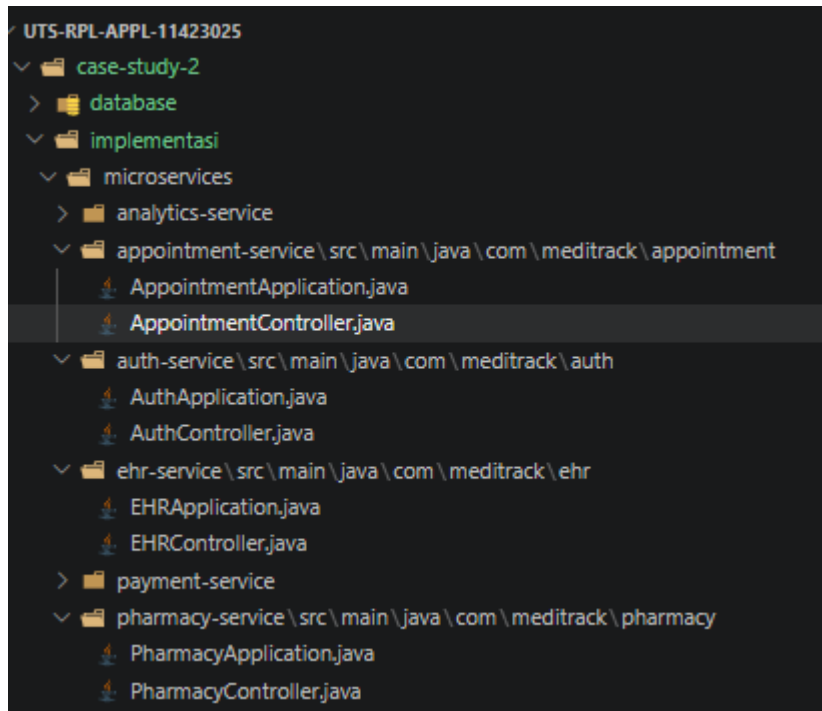
4. Migration Strategy: Strangler Pattern :

Strategi Dekomposisi yang digunakan yaitu strategi Phased Refactoring (Strangler Pattern):

- a. Fase 1: Ekstraksi modul Auth & IAM sebagai pondasi utama keamanan.
- b. Fase 2: Pemisahan Appointment Service karena memiliki trafik tertinggi.
- c. Fase 3: Migrasi modul berat seperti EHR dan Pharmacy.
- d. Fase 4: Implementasi Payment dan Analytics menggunakan event-driven via Kafka.

5. Implementasi (Representasi Kode)

Berikut adalah perbandingannya untuk ditaruh di branch case_study2:



a. Monolith Representation

```
b. package com.meditrack.monolith.controller;
c.
d. import com.meditrack.monolith.service.AppointmentService;
e.
f. public class AppointmentController {
g.     private final AppointmentService appointmentService;
h.
i.     public AppointmentController(AppointmentService
j.         appointmentService) {
k.         this.appointmentService = appointmentService;
l.     }
m.     public String createAppointment(String id, String patientId,
        String doctorId, String date) {
```

```

n.         return appointmentService.createAppointment(id, patientId,
o.         doctorId, date);
p.     }
q. }

```

r. Microservices Mapping (Decoupled)

```

s. package com.meditrack.appointment;
t.
u. import org.springframework.web.bind.annotation.*;
v. import org.springframework.web.client.RestTemplate;
w.
x. @RestController
y. @RequestMapping("/appointments")
z. public class AppointmentController {
aa.
bb.     private RestTemplate restTemplate = new RestTemplate();
cc.
dd.     @PostMapping("/book")
ee.     public String bookAppointment(@RequestBody AppointmentRequest
ff.     request) {
gg.         // Call Auth Service to verify user
hh.         String authUrl = "http://auth-service:8080/auth/users/" +
ii.         request.getPatientId();
jj.         User user = restTemplate.getForObject(authUrl, User.class);
kk.         if (user == null) {
ll.             return "Patient not found";
mm.         }
nn.         // Logic to book appointment
oo.         return "Appointment booked for patient: " +
pp.         user.getUsername();
qq.     }
rr.     @GetMapping("/{id}")
ss.     public Appointment getAppointment(@PathVariable String id) {
tt.         // Logic to get appointment
uu.         return new Appointment(id, "patient1", "doctor1", "2023-10-
vv.         01");
ww.     }
xx. }
yy.
zz. class AppointmentRequest {
    private String patientId;
    private String doctorId;

```

```

aaa.     private String date;
bbb.
ccc.     public String getPatientId() { return patientId; }
ddd.     public String getDoctorId() { return doctorId; }
eee.     public String getDate() { return date; }
fff. }
ggg.
hhh.     class Appointment {
iii.         private String id;
jjj.         private String patient;
kkk.         private String doctor;
lll.         private String date;
mmm.
nnn.         public Appointment(String id, String patient, String doctor,
String date) {
ooo.             this.id = id;
ppp.             this.patient = patient;
qqq.             this.doctor = doctor;
rrr.             this.date = date;
sss.         }
ttt.
uuu.         // Getters
vvv.         public String getId() { return id; }
www.         public String getPatient() { return patient; }
xxx.         public String getDoctor() { return doctor; }
yyy.         public String getDate() { return date; }
zzz.     }
aaaa.
bbbb.     class User {
cccc.         private String id;
dddd.         private String username;
eeee.         private String role;
ffff.
gggg.         public User() {}
hhhh.
iiii.         // Getters
jjjj.         public String getId() { return id; }
kkkk.         public String getUsername() { return username; }
llll.         public String getRole() { return role; }
mmmm.     }

```